SZ

Senzing Bootcamp

Completion Recap

A record of everything you built and learned, module by module.
Keep it, revisit it, and share it with your team.

STARTED
July 16, 2026

PROGRAMMING LANGUAGE
Java

TRACK
Core Bootcamp

MODULES COMPLETED
7 of 7

Modules in this recap

1. Discover the Business Problem
2. SDK Installation and Configuration
3. System Verification
4. Identify and Collect Data Sources
5. Data Quality & Mapping
6. Data Processing
7. Query, Visualize, and Discover

Contents

Module 1: Discover the Business Problem

Completed July 16, 2026

Information Shared

- Entity resolution concepts: principle-based matching, disclosed vs. discovered relationships, explainability, real-time resolution (all sourced from Senzing docs via MCP).
- A gallery of common entity-resolution business patterns (Customer 360, Fraud Detection, Data Migration, Compliance/KYC, Marketing, Healthcare, Supply Chain, Vendor MDM, Insurance), backed by real Senzing use-case and case-study documentation.
- How Senzing's principle-based, entity-level, relationship-aware, explainable matching applies specifically to a multi-source compliance/due-diligence scenario.

Questions & Responses

Q: How much detail would you like in the bootcamp output?

A: standard

Q: Which track would you like?

A: Core Bootcamp

Q: Which programming language would you like to use?

A: Java

Q: Would you like to initialize a git repository?

A: No (skip)

Q: Would you like to see examples of common business problems?

A: Yes

Q: Do any of these patterns match your situation?

A: Requested a generated business scenario instead

Q: Does the generated scenario summary capture your situation accurately?

A: Yes

Q: Will the results need to interface with other software?

A: No

Q: Would you like a data-flow diagram?

A: Yes

Q: Does this accurately capture your problem and approach?

A: Yes

Q: Would you like a one-page executive summary?

A: Yes

Actions Taken

- Generated a bootcamp business scenario (Business Case Offer accepted): Lakeside Community Bank

loan-applicant due diligence, backed by real Senzing Las Vegas CORD data (PPP_LOANS, GLEIF, US-LABOR-VIOLATIONS, OPEN-OWNERSHIP; ~100 records each, ~400 total).

- Created docs/business_problem.md (full problem statement).
- Created config/data_sources.yaml (source registry).
- Created docs/data_flow.md (data flow diagram).
- Created README.md (project overview).
- Created docs/stakeholder_summary.md (executive summary).
- Saved preferences to config/bootcamp_preferences.yaml (verbosity: standard, track: core, language: java, platform: macos/arm64).

Journal

What we did:

Defined the bootcamp's business problem - a compliance/KYC due-diligence scenario for loan applicant vetting - using real Senzing sample data, and documented it for the rest of the bootcamp.

What was produced:

docs/business_problem.md, config/data_sources.yaml, docs/data_flow.md, README.md, docs/stakeholder_summary.md, config/bootcamp_preferences.yaml

Why it matters:

Every later module (SDK setup, mapping, loading, querying) builds directly on this problem statement and these four data sources.

Bootcamper's takeaway:

N/A

Module 2: SDK Installation and Configuration

Completed July 16, 2026

Information Shared

- The Senzing SDK was already installed via Homebrew (version 4.3.2, V4.0+), so installation was skipped; verification and license configuration still ran (both are mandatory).
- Senzing licensing model: the built-in evaluation license allows 500 records automatically (confirmed via MCP); loading record 501 returns SENZ9000. Larger datasets need a custom license at licenses/g2.lic.
- macOS Apple Silicon specifics: DYLD_LIBRARY_PATH must be set before JVM launch; the 4.3 cask can miss rpath entries for sqlite/openssl, so those lib paths were added to the env script.
- SQLite is the correct database for bootcamp learning/eval; the DB file and schema are not auto-created and were created explicitly.
- Never modify global shell config; a project-local env script is used instead.

Questions & Responses

Q: Do you have a Senzing license? (four-option form)

A: 4 - request a free evaluation license now through the bootcamp

Q: Shall I submit the evaluation-license request with those details (Michael / michael@senzing.com / Senzing Bootcamp)?

A: Yes

Q: Which database would you like to use?

A: SQLite

Actions Taken

- Detected existing Senzing SDK 4.3.2 (Homebrew cask) via szBuildVersion.json; skipped install.
- Created SQLite database and schema at database/G2C.db (16 tables).
- Created engine configuration at config/engine_config.json (paths from MCP, DB path overridden to project-local).
- Created project-local environment script src/scripts/senzing-env.sh (SENZING_ROOT, DYLD_LIBRARY_PATH incl. sqlite/openssl, SDK jar, engine config JSON).
- Requested a free 10-day / 250K-record evaluation license via the MCP submit_feedback tool (delivered by email).
- Wrote and ran src/SetupVerify.java: printed SDK version (4.3.2) and active license (EVAL, 500-record limit), registered the default Senzing configuration (config id 1375095766), and confirmed the engine connects to the database.
- Created remaining project directories (licenses/, src/resources/, data/mapping/, data/temp/, build/).
- Recorded license: evaluation in config/bootcamp_preferences.yaml.

Journal

What we did:

Verified the already-installed Senzing SDK works end-to-end in Java - version check, license check, default config registration, and database connection - and configured the project's database, engine config, and environment.

What was produced:

database/G2C.db, config/engine_config.json, src/scripts/senzing-env.sh, src/SetupVerify.java, config/bootcamp_preferences.yaml (license key)

Why it matters:

Every later module (loading data in Module 6, querying in Module 7) depends on a working SDK, a configured database, and the registered default configuration set up here.

Bootcamper's takeaway:

N/A

Module 3: System Verification

Completed July 16, 2026

Information Shared

- The Senzing TruthSet (CUSTOMERS 120, REFERENCE 22, WATCHLIST 17 = 159 records) is the deterministic demo dataset with a published ground-truth key (85 expected entities, 54 known-match clusters), sourced via the MCP server's get_sample_data.
- The Python-based bundled visualization app (scripts/senzing_viz_server.py) requires the Python Senzing SDK, which is unsupported on macOS; a Java-based entity export plus an equivalent Python stdlib/D3 visualization server was used instead, matching the same 4-endpoint contract.
- Entity resolution results: match keys (e.g. "+NAME+DOB+PHONE"), resolution rules (e.g. "CNAME_CFF_CEXCL"), and relationship types (POSSIBLY_SAME, DISCLOSED, AMBIGUOUS) as surfaced in the resolved entity JSON via getEntity(SzRecordKey, SZ_ENTITY_DEFAULT_FLAGS).
- All 10 verification checks and what each one confirms (MCP connectivity, TruthSet acquisition, SDK init, code generation, build, data loading, results validation, database ops, web service, web page).

Questions & Responses

Q: Would you like me to create a quick standalone visualization? (n/a - Module 3 ran normally, Step 9 executed unconditionally)

- Q: Take your time exploring the visualization; let me know when ready.

A: "Done with visualization. continue"

Actions Taken

- Downloaded the 159-record TruthSet (customers.jsonl, reference.jsonl, watchlist.jsonl) and the published ground-truth key (actual_truthset_key.csv: 85 expected entities).
- Wrote and ran src/system_verification/verify_init.java (SDK init check).
- Wrote and ran src/system_verification/verify_pipeline.java: registered CUSTOMERS/REFERENCE/WATCHLIST data sources, loaded all 159 records (0 failures), resolved into 84 entities.
- Wrote and ran src/system_verification/verify_results.java: verified 3/3 known-match clusters resolve correctly, read-by-entity-ID and search-by-attributes both confirmed.
- Wrote src/system_verification/ExportEntityModel.java to export the full resolved-entity model (84 entities, 71 relationships) via getEntity(SzRecordKey, SZ_ENTITY_DEFAULT_FLAGS).
- Wrote src/scripts/senzing_viz_fallback.py: a Python-stdlib + D3 v7 visualization server (4 tabs: Entity Graph, Record Merges, Merge Statistics, Search/Probe) serving the same 4 APIs as the bundled app, built from the Java-exported entity model.
- Generated the standalone snapshot docs/visualizations/truthset_verification.html and ran the live server at port 8080; verified all 4 API endpoints.
- Bootcamper explored the visualization live.
-

Terminated the web service (port 8080 released).

- Purged all 159 TruthSet records via SzEngine.deleteRecord(SzRecordKey) (Java, src/system_verification/PurgeTruthSet.java); confirmed zero TruthSet entities remain.
- Persisted the full Verification Report to config/bootcamp_progress.json.

Journal

What we did:

Ran all 10 system verification checks against the Senzing TruthSet end-to-end - SDK init, code generation, build, data loading, deterministic results validation, database operations, and a live entity-resolution visualization - then cleaned up (stopped the server, purged test data).

What was produced:

src/system_verification/ (verify_init.java, verify_pipeline.java, verify_results.java, ExportEntityModel.java, PurgeTruthSet.java, truthset_data.jsonl, entities_raw.jsonl), src/scripts/senzing_viz_fallback.py, docs/visualizations/truthset_verification.html

Why it matters:

All 10 checks passed - the SDK, database, and configuration from Module 2 are proven to work end-to-end before real project data enters the pipeline in Modules 4-7.

Bootcamper's takeaway:

N/A

Module 4: Identify and Collect Data Sources

Completed July 16, 2026

Information Shared

- The bootcamp's data recommendation hierarchy (CORD first, then free-data repo, then synthesized data) - not needed here since Module 1 already chose CORD.
- The license/dataset-size framing: since the bootcamper's custom license has no record cap (license_record_limit: 0) and the collected total (~400) is far below any threshold, no sampling-for-license or SQLite load-time warning applied.
- Each of the four sources uses a genuinely different schema (flat fields vs. nested NAMES/ADDRESSES arrays), setting up the mapping work in Module 5.

Questions & Responses

Q: I have a data collection checklist that helps you document all your data sources in a structured way. Want me to add it to docs/data_collection_checklist.md?

A: Yes

Actions Taken

- Downloaded full CORD source files for PPP_LOANS (3,488 records), GLEIF (1,952), US-LABOR-VIOLATIONS (1,554), and OPEN-OWNERSHIP (2,039) from the Senzing-hosted URLs.
- Sampled the first 100 records from each into data/raw/ (ppp_loans.jsonl, gleif.jsonl, us_labor_violations.jsonl, open_ownership.jsonl).
- Validated all four files: readable, non-empty, valid JSONL, correct DATA_SOURCE/RECORD_ID - all passed.
- Updated config/data_sources.yaml to the full tracking schema (file paths, record counts, validation status, provenance, mapping/load status).
- Captured config/cord_metadata.yaml (file hashes/sizes) so Module 6 can detect drift before loading.
- Created docs/data_collection_checklist.md and docs/data_source_locations.md.
- Created .gitignore (excludes data/raw/, data/samples/, database/*.db, licenses/*.lic, build artifacts) and docs/security_compliance.md.

Journal

What we did:

Collected the four real CORD data sources planned in Module 1 into the project, validated them, and documented their locations and provenance.

What was produced:

data/raw/ppp_loans.jsonl, data/raw/gleif.jsonl, data/raw/us_labor_violations.jsonl, data/raw/open_ownership.jsonl, config/data_sources.yaml, config/cord_metadata.yaml, docs/data_collection_checklist.md, docs/data_source_locations.md, .gitignore, docs/security_compliance.md

Why it matters:

These four files, each with a different schema, are exactly what Module 5 will assess for quality and map onto the Senzing Entity Specification.

Bootcamper's takeaway:

N/A

Module 5: Data Quality & Mapping

Completed July 16, 2026

Information Shared

- Retrieved the authoritative Senzing Generic Entity Specification (v0.1.0) from the MCP server and saved it to docs/reference/senzing_entity_specification.md as the canonical mapping reference.
- Compared all four sources' fields against the Entity Specification: PPP_LOANS and US-LABOR-VIOLATIONS use flat Senzing attributes (BUSINESS_NAME_ORG/PRIMARY_NAME_ORG, BUSINESS_ADDR_*); GLEIF and OPEN-OWNERSHIP use the nested feature-list structure (NAMES[], ADDRESSES[], IDENTIFIERS[], etc.).
- OPEN-OWNERSHIP is mixed record types (63 ORGANIZATION, 37 PERSON beneficial owners); Senzing reads RECORD_TYPE per record.
- The trade-off of fast-pathing CORD data vs. going through the mapping workflow for hands-on learning: since CORD data is pre-mapped, mapping would re-express an already-correct mapping rather than fix real problems.

Questions & Responses

Q: All four CORD sources are already in Senzing-loadable form. How would you like to proceed? (1) Fast-path all four to loading, or (2) Go through the mapping workflow anyway for learning.

A: 1 (fast-path all four)

Actions Taken

- Downloaded and saved the Senzing Entity Specification to docs/reference/senzing_entity_specification.md.
- Ran the Senzing-readiness check across 100 sampled records per source: all four passed (valid JSON, DATA_SOURCE + RECORD_ID + name feature present in every record).
- Recorded senzing_ready: true for all four in config/data_sources.yaml; corrected OPEN-OWNERSHIP record_type to MIXED.
- Fast-pathed all four sources: set mapping_status: complete and fast_pathed: true in the registry.
- Created docs/mapping/data_lineage.yaml with fast-path lineage entries (source_file == output_file, 100 records in == out, 0 rejected, no transformation script).
- Created docs/data_source_evaluation.md documenting the field-level evaluation and categorization of each source.

Journal

What we did:

Assessed all four data sources against the Senzing Entity Specification, confirmed they are already Entity-Specification-compliant CORD data, and fast-pathed all four directly to loading (no transformation needed).

What was produced:

docs/reference/senzing_entity_specification.md, docs/data_source_evaluation.md, docs/mapping/data_lineage.yaml, updated config/data_sources.yaml

Why it matters:

All four sources are validated and confirmed loadable, so Module 6 can load them directly and run entity resolution across sources.

Bootcamper's takeaway:

N/A

Module 6: Data Processing

Completed July 16, 2026

Information Shared

- Senzing anti-patterns for loading: initialize factory/environment once per process; always drain the redo queue using getRedoRecord() as the loop sentinel (never countRedoRecords(), which does a full table scan); SQLite is dev/test only, not for medium/large production volume.
- Production volume tiers and the license framing: the bootcamper's custom license has no record cap (0), so no capacity concerns regardless of stated production volume.
- The SQLite-at-scale tradeoff for a stated medium production tier: proceed on SQLite for the bootcamp now, migrate to PostgreSQL before real production load (Module 8).
- Why the bootcamp saw 0 PPP_LOANS-to-reference matches: independently-sampled small slices of much larger source files have low odds of specific-company overlap - a sampling-scale artifact, not a pipeline defect, versus the 37 real GLEIF?OPEN-OWNERSHIP matches found (both grounded in the same underlying company registries).

Questions & Responses

Q: How many records do you expect to load in a production system?

A: 3 (500K-10M, medium tier)

Q: SQLite volume heads-up - migrate to PostgreSQL or proceed on SQLite for now?

A: 2 (proceed on SQLite)

Q: Would you like a results dashboard (docs/results_dashboard.html)?

A: No

Q: Would you like a one-page executive summary for this module?

A: No

Actions Taken

- Built src/load/ProductionLoader.java: thread-pool loader (medium-tier architecture) with per-record JSON error logging, progress/throughput reporting, and final statistics - adapted from the MCP-sourced LoadViaFutures.java pattern.
- First load attempt on PPP_LOANS failed 100/100 with SENZ2207 (data source not registered); diagnosed via explain_error_code, fixed by writing and running src/system_verification/RegisterProjectDataSources.java to register PPP_LOANS/GLEIF/US-LABOR-VIOLATIONS/OPEN-OWNERSHIP.
- Re-ran and loaded PPP_LOANS: 100/100, 0 errors, 228 rec/sec; drained 58 redo records via src/load/DrainRedoQueue.java.
- Built src/load/Orchestrator.java: sequential multi-source orchestrator with per-source error isolation, retry with exponential backoff, and health monitoring. Found and fixed an IdentityHashMap "Entry was removed" bug (Map.Entry.getValue() called after Iterator.remove()) during the 10-record sample test, then re-verified before the full run.
- Loaded GLEIF, US-LABOR-VIOLATIONS, OPEN-OWNERSHIP via the orchestrator: 300/300, 0 errors; drained the

coordinated redo queue (0 pending).

- Validated results via src/system_verification/ValidateResults.java: 400 records -> 356 distinct entities, 37 cross-source entities; spot-checked 10 with why-matched review (all verified via matching LEI/name/address, no false positives).
- Documented docs/uat_results.md, docs/results_validation.md, and updated docs/loading_strategy.md with final load order, per-source stats, cross-source summary, and issues/resolutions.
- Updated config/data_sources.yaml: all four sources now load_status: loaded.

Journal

What we did:

Built production-quality single-source and multi-source loading programs, loaded all four data sources into Senzing, processed redo queues, and validated entity resolution results including real cross-source matches.

What was produced:

src/load/ProductionLoader.java, src/load/DrainRedoQueue.java, src/load/Orchestrator.java, src/system_verification/RegisterProjectDataSources.java, src/system_verification/ValidateResults.java, docs/uat_results.md, docs/results_validation.md, updated docs/loading_strategy.md and config/data_sources.yaml

Why it matters:

All 400 records are now resolved in the Senzing database with validated match quality - Module 7 can query, visualize, and explore these results against the original business problem.

Bootcamper's takeaway:

N/A

Module 7: Query, Visualize, and Discover

Completed July 16, 2026

Information Shared

- Query requirements were derived directly from Module 1's success criteria (correct resolution without manual cross-checking, why-matched explanations, low false positives) and desired output (per-applicant due-diligence reports).
- Matching-concepts refresher applied to the bootcamper's own data: match keys (e.g. +NAME+ADDRESS+LEI_NUMBER), confidence via SZ_INCLUDE_MATCH_KEY_DETAILS, and real cross-source connections (GLEIF <-> OPEN-OWNERSHIP via shared LEI numbers).
- Quality evaluation methodology (entity-to-record ratio, possible-match rate, cross-source match rate) computed directly from the exported entity model since no data mart was built at this scale.
- Honest finding: 0 of 100 PPP_LOANS applicants matched a reference source in this run - a sampling-scale artifact (small independent samples of much larger source files), not a pipeline defect, contrasted with 37 real reference-to-reference matches found (36 GLEIF<->OPEN-OWNERSHIP, 1 GLEIF<->US-LABOR-VIOLATIONS: Wynn Las Vegas LLC).

Questions & Responses

Q: Is there anything you'd like to adjust to the derived query requirements?

A: Add a visualization like in Module 3

Q: Would you like a results dashboard saved as a standalone page?

A: Yes

Q: Would you like to explore the Discover phase, or skip straight to wrapping up Module 7?

A: Skip and wrap up

Actions Taken

- Derived 5 query requirements from docs/business_problem.md success criteria plus the bootcamper's added visualization requirement.
- Built src/query/DueDiligenceReport.java: per-applicant report using getEntity(SzRecordKey, flags) with SZ_ENTITY_DEFAULT_FLAGS + SZ_INCLUDE_MATCH_KEY_DETAILS; iterates over loaded PPP_LOANS records (never guessed entity IDs). Fixed a flag-type compile error (SZ_ENTITY_INCLUDE_ALL_RELATIONS alone lacks entity names) by switching to SZ_ENTITY_DEFAULT_FLAGS.
- Built src/query/SearchApplicant.java: search-by-attributes lookup; verified against "Bally Technologies" (found the correct GLEIF entity).
- Ran both programs successfully: 0/100 applicants flagged (explained honestly, not fabricated).
- Computed quality indicators (entity-to-record ratio 0.89, possible-match rate 1.4%, cross-source match rate 10.4%) directly from an exported entity model (data/mapping/project_entities.jsonl, 356 entities) since no data mart exists at this scale; assessed as Acceptable.
- Generated the entity graph visualization (Module 3-style): standalone snapshot

docs/visualizations/due_diligence_results.html plus a live server (verified all 4 API endpoints), then cleanly stopped it.

- Identified a genuine cross-reference: Wynn Las Vegas LLC matched GLEIF <-> US-LABOR-VIOLATIONS on +NAME+ADDRESS.
- Built docs/results_dashboard.html: a static findings dashboard (KPIs, per-source stats, cross-source overlap, entity size distribution, quality indicators, top match keys, key findings).
- Discover phase (advanced why/how/network tour) explicitly skipped by the bootcamper.

Journal

What we did:

Built and ran query programs that directly answer Lakeside Community Bank's due-diligence question, evaluated entity resolution quality, and produced two visualizations of the results (an interactive entity graph and a static findings dashboard).

What was produced:

src/query/DueDiligenceReport.java, src/query/SearchApplicant.java, data/mapping/project_entities.jsonl, docs/visualizations/due_diligence_results.html, docs/results_dashboard.html

Why it matters:

This is the payoff of the entire bootcamp - the original business problem (manual cross-checking of loan applicants against reference lists) now has working, tested query programs and visualizations built directly on top of validated entity resolution results.

Bootcamper's takeaway:

N/A